

Title: Displaying Strings and Numbers on the LCD of the 8086 Microprocessor Kit in PC Mode.

Abstract:

This experiment was performed using the Intel 8086 Microprocessor Trainer Kit in PC Mode to display messages on a 16x2 LCD by using assembly language instructions. The main objective of this experiment was to understand how the 8086 microprocessor communicates with an LCD module through specific instruction, status, and data ports, and how characters are displayed line by line by sending command bytes and ASCII data to the LCD. In this experiment, the LCD was first cleared, the first message "Serial Monitor!!" was displayed on the first line, the cursor was moved to the second line, and the second message "We Are SUM26" was displayed there. After that, the display was shifted left repeatedly and the whole sequence was executed continuously. The experiment helped to understand LCD interfacing, busy-flag checking, port addressing, delay generation, memory-based data access, and practical hardware control using the 8086 microprocessor.

Introduction (Theory):

The Intel 8086 is a 16-bit microprocessor that communicates with external hardware devices through Input/Output (I/O) ports. Output devices such as LEDs, seven-segment displays, and LCD modules are commonly interfaced with the 8086 in laboratory experiments. By using the OUT instruction, the microprocessor can send commands or data to an external device, and by using the IN instruction it can read status information from that device.

A 16x2 character LCD is an electronic display device that can show alphanumeric characters on two lines. The LCD must receive command bytes for operations such as clearing the display, moving the cursor, and shifting the displayed text. It must also receive character data bytes to show letters, spaces, and symbols. In this experiment, three LCD-related ports were used: LCDC for sending commands, LCDC_S for reading the status or busy flag, and LCDD for sending character data.

The program begins by setting up the stack and then repeatedly performs the same display sequence. The ALLCLR subroutine clears the LCD. The STRING subroutine reads characters one by one from memory by using the SI register and sends them to the LCD until the terminating value 00H is found. The LN21 subroutine moves the cursor to the second line by sending the command C0H. The SHIFT subroutine sends the command 00011100B to shift the display to the left. Before every command or character transfer, the BUSY subroutine checks the busy flag of the LCD to make sure that the device is ready. Therefore, this experiment demonstrates practical LCD control, memory addressing, repeated execution, and real-time hardware interfacing using the 8086.

Material:

Hardware	Software
1. Intel 8086 Microprocessor Trainer Kit 2. 16x2 LCD Display Module 3. Personal Computer (PC) 4. Serial / Interface Cable	1. Microsoft Macro Assembler (MASM) 2. MDA-8086 / MDA-WinIDE8086 3. Text Editor (Notepad) 4. Command Prompt (CMD)

Method:

Port and Subroutine Summary:

Symbol	Value	Purpose
LCDC	00H	LCD instruction port used to send commands such as clear, line select and shift
LCDC_S	02H	LCD status port used to read the busy flag before data transfer
LCDD	04H	LCD data port used to send character data to the display
ALLCLR	01H command	Clears the entire LCD display
LN21	C0H command	Moves the cursor to the second line of the LCD
SHIFT	1CH command	Shifts the displayed text left by one position

Code:

```
CODE SEGMENT
    ASSUME CS:CODE, DS:CODE, ES:CODE, SS:CODE
    ;
STACK EQU 0540H
    ;
LCDC EQU 00H
LCDC_S EQU 02H
LCDD EQU 04H
    ;
    ORG 1000H
    ;
    XOR AX,AX
```

```

MOV SS, AX
MOV SP, STACK
;
CALL ALLCLR
; MOV SI, OFFSET CUSOR1
CALL STRING
;
CALL LN21
MOV SI, OFFSET CUSOR2
CALL STRING
;
L1: CALL DISPOFF
CALL TIMER
CALL DISPON
CALL TIMER
JMP L1
;
CUSOR1 DB 'Serial Monitor!!', 00H, 00H
CUSOR2 DB 'We Are SUM26', 00H, 00H
;
; LCD instruction
ALLCLR; MOV AH, 01H
JMP LNXX
;
DISPOFF:
MOV AH, 08H
JMP LNXX
;
DISPON: MOV AH, 0FH
JMP LNXX
;
LN11: MOV AH, 02H
JMP LNXX
;
LN21: MOV AH, 0C0H
;
LNXX: CALL BUSY
MOV AL, AH
OUT LCDC, AL
RET
; busy flag check
BUSY: IN AL, LCDC_S
AND AL, 10000000B
JNZ BUSY
RET
;
; 1 char, LCD OUT
; AH = out data
CHAROUT:
CALL BUSY

```

```

;
MOV AL,AH
OUT LCDD, AL
RET
;
STRING: MOV AH, BYTE PTR CS:[SI]
        CMP AH,00H
        JE STRING1
        ; out
        CALL BUSY
        CALL CHAROUT
        INC SI
        JMP STRING
STRING1:
        RET
;
TIMER: MOV CX, 2
TIMER2: PUSH CX
        MOV CX, 0
TIMER1: NOP
        NOP
        NOP
        NOP
        LOOP TIMER1
        POP CX
        LOOP TIMER2
        RET
;
CODE ENDS
END

```

Execution:

1. First, the assembly language program was written in Notepad and saved as .asm file.
2. Machine code execution using CMD-

```

C:\Users\diu>cd..
C:\Users>cd..
C:\>cd MDA
C:\MDA>cd 8086
C:\MDA\8086>cd Asm8086
C:\MDA\8086\Asm8086>MASM.exe sum26_2.asm

```

Object filename [SUM26_1.OBJ]: sum26_2

Source listing [NUL.LST]: sum26_2

Cross-reference [NUL.CRF]:

0 Warning Errors

0 Severe Errors

3. The MDA (Microprocessor Development and Analysis) software was opened.

4. The generated assembly program file was loaded using the File → Open option.
5. The program was assembled and linked to generate the .ABS (Absolute) file using the Assemble & Link option.
6. The generated .ABS file was loaded into the simulator.
7. The program was executed using the Run command.
8. The output and register values were observed through the simulator window and LCD display panel.

Result:

Observed Output Sequence:



Fig 1: LCD Interface Displaying Serial Data.

Discussion:

This experiment showed how an 8086 assembly language program can be developed and executed in PC Mode using MDA software. The source program was assembled and linked to create an ABS file, which was then run in the simulator. Using PC Mode made the process easier and faster because the program did not need to be entered manually in machine code.

The program successfully displayed a message on the 16×2 LCD. Different subroutines were used to perform specific tasks. The BUSY subroutine checked whether the LCD was ready to receive data, the STRING subroutine displayed the stored message character by character, and the SHIFT subroutine moved the text across the display with a delay.

Through this experiment, we gained practical knowledge about assembly language programming, LCD interfacing, program execution in a simulator, and handling strings stored in memory. It also helped us understand how software can be used to control hardware devices in a microprocessor system.